Rigid-Body Ackermann Model: A Review

Lucas Libshutz
Cornell University, Mechanical and Aerospace Engineering

***Abstract*—For modern autonomous driving of high speed vehicles, high-accuracy localization is an essential requirement for proper control and planning, especially in competition environments. More specifically, for the Shell Eco Marathon, a car that is going 20+ MPH and on an angled track needs to have high confidence in both angle and position in order to control the car properly on a banked track. This work aims to provide a rigid body based EKF that can provide the orientation and position data to a high level of accuracy, while maintaining model fidelity and speed at runtime. Using JAX to formulate automatically differentiable dynamics, the EKF runs at *0.5 ms* for each predict + update iteration, and results in an average RMSE of approximately *2.8 cm* and *3.5 cm/s* for position and velocity respectively.**

I. Goal

One of the main motivations for this project was to create a base on which the team could test and run an autonomous car in a low stakes environment, and use that to tune the kinematic parameters and the dynamic assumptions for the larger, more complex vehicle that would be tested afterwards. Because of the complex nature of the dynamics of the larger car, some of the main aspects of the system that were modeled included:

- Contact dynamics in z of the tires and the ground
- Orientation and tilt of the vehicle in roll, pitch, *and* yaw
- Tire slip parameters in both lateral and longitudinal directions

To do the last part is especially important for a larger car in a variable driving environment, where variable amounts of tire slip can drastically impact the localization accuracy of a filter, and contribute to large amounts of uncertainty.

II. Derivation of Relevant Dynamics

A. Notation

- $\mathcal{I}$ denotes an intertial frame, with its z axis pointing in the direction against gravity.
- The position vector in frame $\mathcal{A}$ to a point Q relative to the frame origin is denoted as ${}^{\mathcal{A}}\mathbf{p}_Q$.
- ${}^{\mathcal{A}}R_{\mathcal{B}}$ denotes the rotation matrix from frame $\mathcal{B}$ to frame $\mathcal{A}$. This is such that ${}^{\mathcal{A}}\mathbf{p} = {}^{\mathcal{A}}R_{\mathcal{B}}{}^{\mathcal{B}}\mathbf{p}$.
- Let $[\mathbf{x}]_\times \in \mathbb{R}^{3\times 3}$ be the skew-symmetric matrix such that $[\mathbf{x}]_\times \mathbf{y} = \mathbf{x} \times \mathbf{y}$, where $\times$ is the cross product operator in $\mathbb{R}^3$.

B. System Dynamics Formulation

With the above notation in mind, we formulate the state of the vehicle as a whole as the following vector:

$$\mathbf{x} = \begin{bmatrix} \mathbf{p} \in \mathbb{R}^3 \\ R \in SO(3) \\ \mathbf{v} \in \mathbb{R}^3 \\ {}^{\mathcal{B}}\boldsymbol{\omega} \in \mathbb{R}^3 \\ \boldsymbol{\omega}_w \in \mathbb{R}^4 \end{bmatrix}$$

And the corresponding control input is also defined as:

$$\mathbf{u} = \begin{bmatrix} \delta \in \mathbb{R} \\ \boldsymbol{\tau} \in \mathbb{R}^2 \end{bmatrix}$$

From this, we now derive the full state space model of the system. First, we define the translational and rotional kinematics of the car as a whole via Newton's second law:

$$\begin{aligned} m\dot{\mathbf{v}} &= m\mathbf{g} + \sum_i \mathbf{f}_i \\ \dot{\mathbf{p}} &= \mathbf{v} \end{aligned} \tag{1}$$

Where $\mathbf{f}_i \in \mathbb{R}^3$ comes from the wheel dynmaics, detailed later. And similarly, for rotational kinematics:

$$\begin{aligned} \mathbb{I}_G{}^{\mathcal{B}}\dot{\boldsymbol{\omega}} + {}^{\mathcal{B}}\boldsymbol{\omega} \times \left(\mathbb{I}_G{}^{\mathcal{B}}\boldsymbol{\omega}\right) &= \boldsymbol{\tau}(\mathbf{x}, \mathbf{u}) \\ \dot{R} &= R[{}^{\mathcal{B}}\boldsymbol{\omega}]_\times \end{aligned} \tag{2}$$

And the formulation for the update of the orientation uses the built-in `jaxlie.SO3.exp` formulation, which evolves on $SO(3)$ while using a quaternion ($q \in S^3$) representation internally within the `jaxlie` library.

The quantity $\mathbf{f}_i$ is the force generated by the i-th wheel, and is calculated by both contact dynamics in z and slip dynamics in both x and y. To calulate the wheel dynamics, we first start wth the normal force $f_{z,i}$:

$$f_{z,i} = k_n \cdot \mathrm{ReLU}(z_0 - z) + c_n \cdot \mathrm{ReLU}(0 - v_z) \tag{3}$$

where $f_{z,i}$ is enforced to be positive, such that the car is not pulled into the ground (i.e. $z_0 - z > 0$). z_0 is the initial height, and z is the current height of the wheel. v_z is the vertical velocity of the wheel, and k_n and c_n are the stiffness and damping coefficients for the normal force, respectively.

Then, the wheel slip *ratios* are then found via the Pacejka slip model, which is a commonly used empirical model for tire slip dynamics. The slip ratios are defined as follows:

$$\kappa_i = \kappa_{\max} \cdot \frac{r\omega_i - v_{t,i}}{\sqrt{v_{t,i}^2 + \epsilon^2}}$$
$$\alpha_i = \arctan\left(\frac{v_{n,i}}{\sqrt{v_{t,i}^2 + \epsilon^2}}\right) \tag{4}$$

In equation (4), the tolerance ϵ is added to prevent division by zero when the tangential velocity $v_{t,i}$ is very small. Furthermore, κ and α are also both bounded by a $\tanh$ function each, in order to bound large slip changes that may induce negative RPM values in the model. The slip ratios κ_i and α_i are then used to calculate the longitudinal and lateral forces generated by the wheel, via $f_{x,i}^\star = C_\kappa \cdot \kappa$ and $f_{y,i}^\star = -C_\alpha \cdot \alpha$, where C_κ and C_α are the longitudinal and lateral slip stiffness coefficients, respectively. From this, all forces are scaled by the maximum normal force, formulated as:

$$f_{\max} = \mu \cdot f_{z,i}$$
$$\text{scale} = \frac{f_{\max}}{\sqrt{\left(f_{x,i}^\star\right)^2 + \left(f_{y,i}^\star\right)^2 + \left(f_{\max}\right)^2}} \tag{5}$$

From this, both $f_{x,i}^\star$ and $f_{y,i}^\star$ are scaled by the above quantity, such that the total force generated by the wheel is bounded by the maximum friction cone specified by μ. Finally, the forces are then rotated into the inertial frame and summed across all wheels to get the total force on the car, which is then used in the translational dynamics of the car as shown in equation (1).

The one other aspect of the dynamics that remains is the spin dynamics of each wheel specifically, which we define as a damped rotating disk as follows:

$$I_w \dot{\omega}_{w,i} = \tau_{\text{cmd},i} - r f_{x,i} - b_w \omega_{w,i} \tag{6}$$

Simiarlly, for the passive wheels on the car (i.e. front wheels for RWD), the dynamics are simplified to relax toward the standard kinematic rolling constraint:

$$\dot{\omega}_{w,i} = \frac{1}{\tau_f}\left(\frac{v_{t,i}}{r} - \omega_{w,i}\right) \tag{7}$$

The full state update blends the wheel rotation dynamics via a motor mask, such that $\dot{\omega}_w = \texttt{mask} \cdot \dot{\omega}_{\text{drive}} + (1 - \texttt{mask}) \cdot \dot{\omega}_{\text{passive}}$.

All combined together, the system of equations that evolve our state *cannot* be formed into a linear state-space model such as $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$, due to the nonlinearities in the slip model and the rotation dynamics. However, the system can be linearized around a nominal trajectory, which is what is done in the next section.

III. Extended Kalman Filter Formulation

A. Error State Formulation

While most of the properties of the state $\mathbf{x}$ can be updated via the form $\mathbf{x} \leftarrow \mathbf{x} + K\nu$, we cannot do the same for the orientation R, since it evolves on the manifold $SO(3)$. If we ran a standard EKF with a quaternion directly in the state vector, the additive update would push q off of the unit sphere required to properly represent rotations.

In order to solve this problem, this work formulates a filter in **error state**. This means that instead of tracking full $SO(3)$, we instead formulate the error orientation as $\delta\boldsymbol{\theta} \in \mathbb{R}^3$, which lives in the tangent space $\mathfrak{so}(3) \cong \mathbb{R}^3$. Doing so allows us to implement a standard state update into the error state EKF, whose state is defined as:

$$\delta\mathbf{x} = \begin{bmatrix} \delta\mathbf{p} \in \mathbb{R}^3 \\ \delta\boldsymbol{\theta} \in \mathbb{R}^3 \\ \delta\mathbf{v} \in \mathbb{R}^3 \\ \delta^{\mathcal{B}}\boldsymbol{\omega} \in \mathbb{R}^3 \\ \delta\boldsymbol{\omega}_w \in \mathbb{R}^4 \end{bmatrix}$$

where all updates for non-orientation states are standard, except for $\delta\boldsymbol{\theta}$, which is used to update the orientation as follows:

$${}^{\mathcal{B}}R_{\mathcal{I}}+ \leftarrow {}^{\mathcal{B}}R_{\mathcal{I}} \cdot \exp\left([\delta\boldsymbol{\theta}]_\times\right) \tag{8}$$

B. Filter Prediction

As the components of the system are nonlinear, as stated before we do not have a standard Kalman filter predict step; rather, we propagate the nominal state $\hat{\mathbf{x}}_{k+1|k}$ as:

$$\hat{\mathbf{x}}_{k+1|k} = f(\hat{\mathbf{x}}_{k|k}, \mathbf{u}_k) \tag{9}$$

where f is the integration step of the `CarModel` class, implemented in the library. Similarly for the covariance, we do not have a standard $P_{k+1|k} = FP_{k|k}F^T + Q$ update, but rather we linearize the dynamics around the nominal trajectory to get the Jacobian for the error dynamics, as:

$$F_k = \left.\frac{\partial f}{\partial \delta\mathbf{x}}\right|_{\hat{\mathbf{x}}_{k|k}, \mathbf{u}_k} \tag{10}$$

And this then feeds into the covariance propagation as:

$$P_{k+1|k} = F_k P_{k|k} F_k^T + Q \tag{11}$$

As $\delta\mathbf{x}$ is reset to zero after each error injection, the prediction error state mean will *always be zero*, as the predict step will only propagate covariance.

C. Filter Update